Flip Array
Electronics-ICT

(B-KUL-JLI1QS)

Arne Vanmarcke, Thibault Dekock

February 10, 2025



Instructor: Dr. Thomas Feys

1 Introduction

The first exercise is simple: flip an array on the GPU using multiple blocks with multiple threads (Figure: 1).

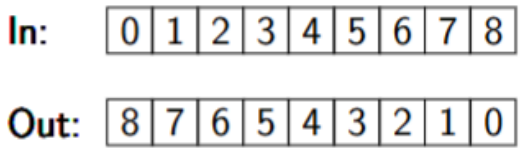


Figure 1: Flipped array

2 Tests

he maximum block size is 1024 threads per block. When using 5000 threads per block, the execution fails and returns zeros (Figure 2).

```
Flipped array GPU: 0 0 0 0 0 0 0 0 0
Execution time GPU: 255 microseconds
```

Figure 2: 5000 threads

With one thread and three blocks, only three numbers change (Figure 3). Adding another thread results in six changes, confirming that the total number of operations is limited to *threads* $\times$ *blocks*.

```
Flipped array GPU: 8 7 6 0 0 0 0 0 0
Execution time GPU: 341 microseconds
```

Figure 3: Limited threads

Increasing the number of blocks to 1000 while using three threads decreases execution speed (Figure 4).

```
Flipped array GPU: 8 7 6 5 4 3 2 1 0
Execution time GPU: 392 microseconds
```

Figure 4: 1000 blocks

The CPU processes the flipped array significantly faster (Figure 5).

```
Flipped array GPU: 8 7 6 5 4 3 2 1 0
Execution time GPU: 352 microseconds
Flipped array CPU: 8 7 6 5 4 3 2 1 0
Execution time CPU: 23 microseconds
```

Figure 5: Results

3 Code

This program flips an array on both the GPU and CPU and compares execution times. The flipArray kernel runs on the GPU, where each thread swaps one element. Memory is allocated, data is copied to the GPU, and the kernel executes with blocksPerGrid = 3 and threadsPerBlock = 3. Results are copied back to the CPU and printed. Execution times are measured using std::chrono.

```cpp
#include <cuda_runtime.h>
#include <stdio.h>
#include <chrono>

using namespace std::chrono;

__global__ void flipArray(int *d_in, int *d_out, int n) {
    //bereken array index
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n) {
        d_out[idx] = d_in[n - 1 - idx];
    }
}

void flipArrayCPU(int* in, int* out, int n) {
    for (int i = 0; i < n; i++) {
        out[i] = in[n - 1 - i];
    }
}

int main(){
    const int n = 9;
    int h_in[n] = {0, 1, 2, 3, 4, 5, 6, 7, 8};
    int h_out[n];

    int *d_in, *d_out;
    size_t size = n * sizeof(int);

    cudaMalloc((void**)&d_in, size);
    cudaMalloc((void**)&d_out, size);

    // kopieer array naar gpu
    cudaMemcpy(d_in, h_in, size, cudaMemcpyHostToDevice);

    //bereken dimensies
    int threadsPerBlock = 3;
    int blocksPerGrid = 3;

    //voer het flippen uit
    auto start = high_resolution_clock::now();

    flipArray<<<blocksPerGrid, threadsPerBlock>>>(d_in, d_out, n);

    auto stop = high_resolution_clock::now();

    auto duration = duration_cast<microseconds>(stop - start);

    //kopieer resultaten naar host
    cudaMemcpy(h_out, d_out, size, cudaMemcpyDeviceToHost);

    printf("Flipped array GPU: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", h_out[i]);
    }
    printf("\n");
    printf("Execution time GPU: %d microseconds\n", duration.count());

    cudaFree(d_in);
    cudaFree(d_out);

    start = high_resolution_clock::now();

```

```
63      flipArrayCPU(h_in, h_out, n);
64
65      stop = high_resolution_clock::now();
66
67      duration = duration_cast<microseconds>(stop - start);
68
69      printf("Flipped array CPU: ");
70      for (int i = 0; i < n; i++) {
71          printf("%d ", h_out[i]);
72      }
73      printf("\n");
74      printf("Execution time CPU: %d microseconds\n", duration.count());
75
76      return 0;
77  }
```